

Grado en Ingeniería Informática

Tecnologías de Desarrollo de Software

Presentación de la asignatura



Departamento
Informática y
Sistemas

Índice

Objetivos

Contenidos de Teoría y Prácticas

Planificación de clases

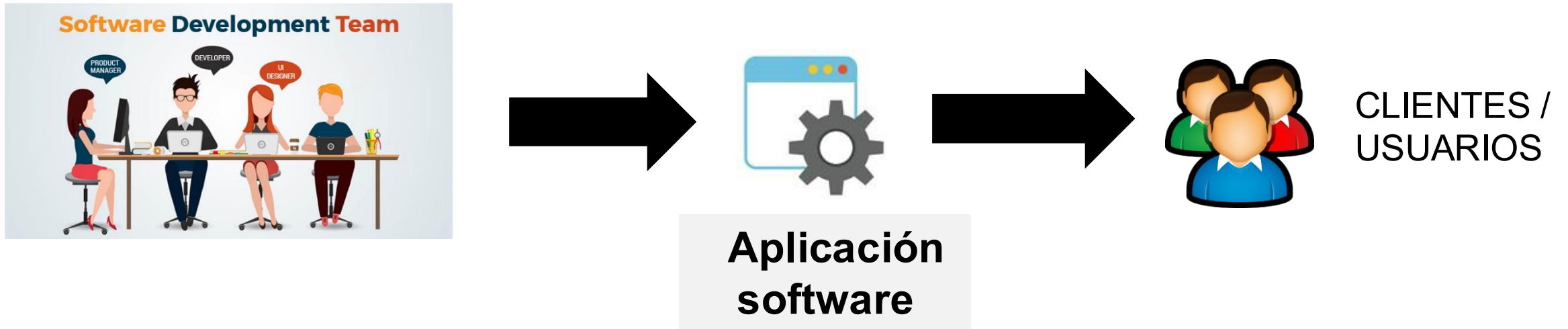
Evaluación

Carga estimada para el alumno

Bibliografía

Aplicaciones software

- Una *aplicación software* realiza una funcionalidad que permite automatizar una tarea en un determinado dominio o área de trabajo.



- Su *tamaño y complejidad* es variable y determina el tiempo + esfuerzo requerido en su construcción.
 - **Métricas:** LOC (líneas de código), FP (puntos de función), etc. - - > *personas/año*



70 % 10:09



Instagram



Historias

▶ Ver todas



Tú



mkbhd



paoloac



jacampon



ma



joanleon

Caldas De Malavella, Cataluna, Spain



prime video

EN



Welcome to Prime Video

Enjoy exclusive Amazon Originals as well as popular movies and TV shows. Join Prime Video now for USD 5.99 per month. Cancel anytime.

Prime Video Member? Sign in

or

Start your free trial*

Membership renews at USD 5.99/month.

AMAZON ORIGINAL



AMAZON ORIGINAL



AMAZON ORIGINAL



AMAZON ORIGINAL



AMAZON ORIGINAL



AMAZON ORIGINAL



AMAZON ORIGINAL



AMAZON ORIGINAL



AMAZON ORIGINAL



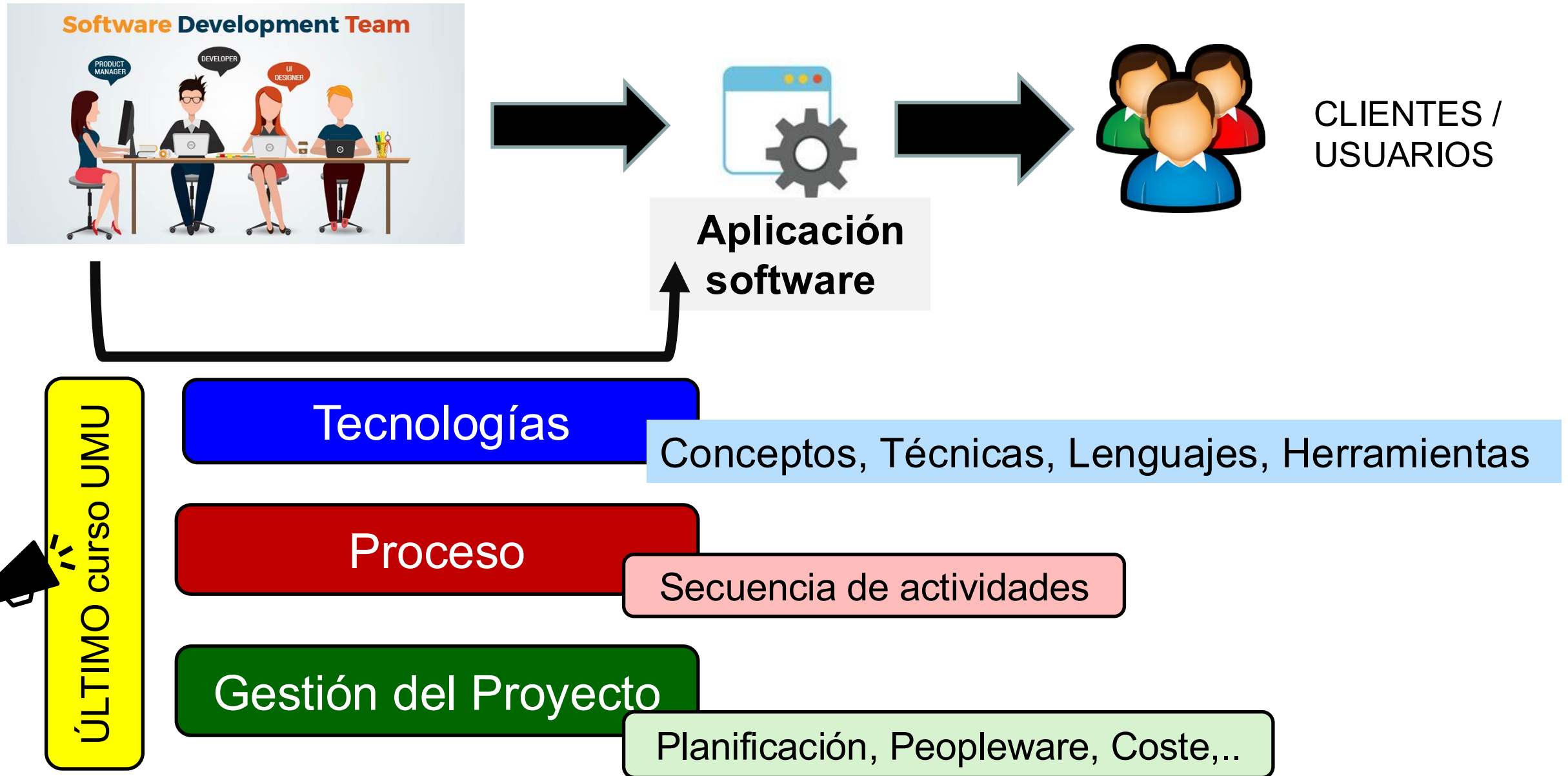
Aplicaciones software

- Una *aplicación software* realiza una funcionalidad que permite automatizar una tarea en un determinado dominio o área de trabajo.



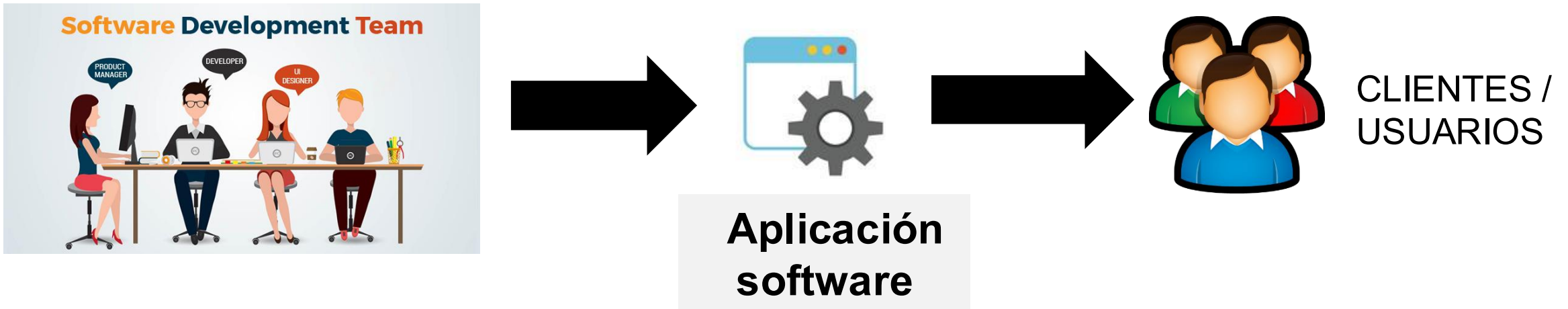
- Implica a un **equipo de desarrolladores** y seguir un **proceso no trivial** que comprende desde la captura de los requisitos hasta el despliegue y mantenimiento de la aplicación, y uso de una arquitectura, diseño de UI, gestión del equipo, entre otras tareas.

Construcción de Software



Aplicaciones software

- Una *aplicación software* realiza una funcionalidad que permite automatizar una tarea en un determinado dominio o área de trabajo.



Ingeniería del software es la disciplina que se ocupa de todas las cuestiones relacionadas con el desarrollo de software aplicando principios, métodos y técnicas propios de un proceso ingenieril.

Punto de partida de los alumnos: Algoritmos y pequeños programas

OPTIMAL-BST(p, q, n)

```

1  let  $e[1..n+1, 0..n]$ 
    and  $root[1..n, 1..n]$  be new tables
2  for  $i = 1$  to  $n + 1$ 
3       $e[i, i - 1] = q_{i-1}$ 
4       $w[i, i - 1] = q_{i-1}$ 
5  for  $l = 1$  to  $n$ 
6      for  $i = 1$  to  $n - l + 1$ 
7           $j = i + l - 1$ 
8           $e[i, j] = \infty$ 
9           $w[i, j] = w[i, j - 1] + p_j + q_j$ 
10         for  $r = i$  to  $j$ 
11              $t = e[i, r - 1] + e[r + 1, j] + w[i, j]$ 
12             if  $t < e[i, j]$ 
13                  $e[i, j] = t$ 
14                  $root[i, j] = r$ 
15 return  $e$ 

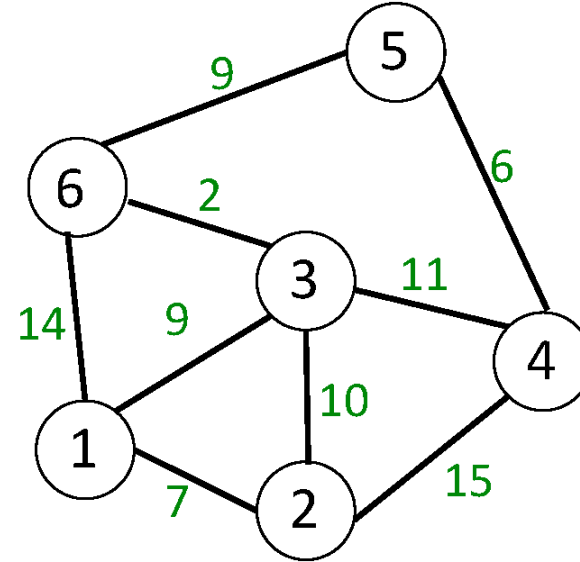
```

```

1  SELECT DISTINCT sale_date, store_state,
2      sum(sale_amount) as total_sales
3  FROM customers
4  GROUP BY store_state, sale_date
5  ORDER BY sale_date ASC;

```

	Data Output	Messages	Notifications	Explain
	sale_date date	store_state character varying (255)	total_sales numeric	
1	2020-05-21	NY	1200	
2	2020-05-22	MH	1500	
3	2020-05-22	NULL	2410	
4	2020-05-23	KA	1200	
5	2020-05-23	MH	2520	



```

package ren
import java.....,
import java.util.Vector;

class Customer {
    private String _name;
    private Vector<Rental> _rentals = new Vector<Rental>();

    public Customer(String name) {
        _name = name;
    }

    public String getMovie(Movie movie) {
        Rental rental = new Rental(new Movie("", Movie.NEW_RELEASE), 10);
        Movie m = rental._movie;
        return movie.getTitle();
    }

    public void addRental(Rental arg) {
        _rentals.addElement(arg);
    }

    public String getName() {
        return _name;
    }
}

```

[Kindle Store](#) › [Kindle eBooks](#) › [Computers & Technology](#)

Design Patterns: Elements of Reusable Object-Oriented Software 1st Edition, Kindle Edition

by [Gamma Erich](#) (Author), [Helm Richard](#) (Author), [Johnson Ralph](#) (Author), [Vlissides John](#) (Author), [Grady Booch](#) (Foreword) | Format: Kindle Edition

★★★★★ ▾ 1,922 ratings

[Look inside](#) ▾

ISBN-13: 978-0201633610

ISBN-10: 0201633612

[Why is ISBN important?](#) ▾[Kindle](#)   [Hardcover](#) [Paperback](#) [Other Sellers](#)

¿Qué conocimientos básicos
son necesarios para
construir aplicaciones
software ?

Tablet devices. [See all supported devices](#) ▾

Sold by: Amazon.com Services LLC

\$25.43

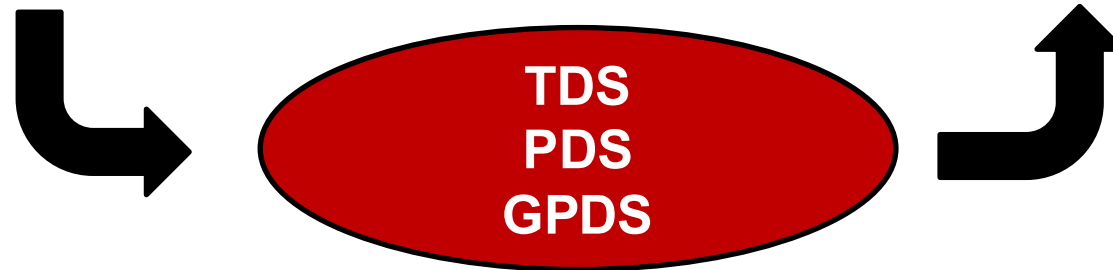
includes VAT

Digital List Price: \$25.74 Save \$0.31 (1%)

[Buy now with 1-Click®](#)[Send a free sample](#)[Deliver to your Kindle or other device](#)[Buy for others](#)[Give as a gift or purchase for a team or group.](#)[Learn more](#) ▾[Buy for others](#)[Enter a promotion code or Gift Card](#)

Ingeniería del Software en la FIUM (último curso)

Algoritmos
Estructuras de datos
Fundamentos OO
Concurrencia



“Ir equipándose” para aplicar técnicas de **ingeniería de software** en el desarrollo de aplicaciones.

Ingeniería del Software en la FIUM (último curso)

Grado Ing. Informática (Tercero)

Cuatrimestre 1

Tecnologías de Desarrollo de Software

Cuatrimestre 2

Procesos de Desarrollo de software

Gestión de Proyectos Software

Mención Ing. Software (Cuarto curso)

Cuatrimestre 1

Aplicaciones Distribuidas

Calidad del Software

Interfaces de Usuario

Programación Paralela

Informática Gráfica

Cuatrimestre 2

Desarrollo de Aplicaciones Web

Arquitectura Software

Sistemas Multimedia

Master Ing. Software

Cuatrimestre 1

Desarrollo Full-Stack

Calidad y Pruebas del Software

Prácticas Continuas (DevOps)

Gestión Ágil de Proyectos

Bases de datos NoSQL

Cuatrimestre 2

Desarrollo Full Stack

Desarrollo en la nube

Software Seguro

IA aplicada al desarrollo de software

Generación Automática de Código

TDS: Objetivo principal

- El alumno adquirirá **competencias básicas para iniciar su formación** en la práctica del desarrollo de software.
- El alumno aprenderá **conceptos, técnicas y herramientas** básicos en construcción de aplicaciones.

Conceptos / Técnicas	Herramientas
Ciclo de Vida del Desarrollo de Software, Procesos Software, Requisitos	
Modelado Software: Diagramas de Clases, Interacciones, Máquinas de Estado con UML	Editor UML
GRASP , DDD, programación funcional en Java, introspección, patrones de diseño	Java >= 8 , Eclipse IDE , LLMs, Stackoverflow ...
Arquitectura en capas	
Creación de interfaces de usuario, GUI	JavaFX , Scene Builder
Control de versiones	Git
Construcción de proyectos Java	Maven
Depuración	JDT en Eclipse

TDS: Competencias (Teoría)

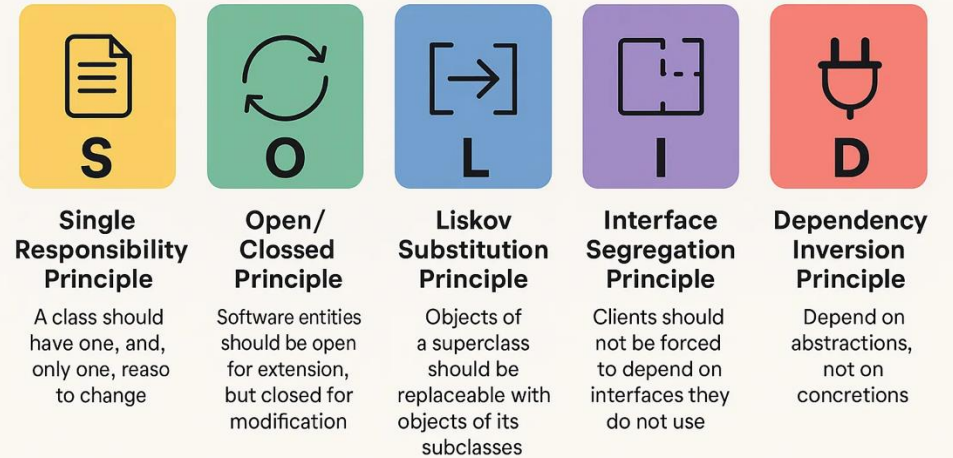
■ Introducción al Desarrollo del Software

- Ciclo de vida: Etapas y Modelos de procesos
- Requisitos
- Modelado con UML

■ Principios básicos en construcción de software OO

- Patrones **GRASP**: Experto, Controlador y Creación
- Principio Separación Modelo-Vista
- Arquitectura limpia y MVC
- Principios **SOLID**
- Principio de Inversión de Dependencias

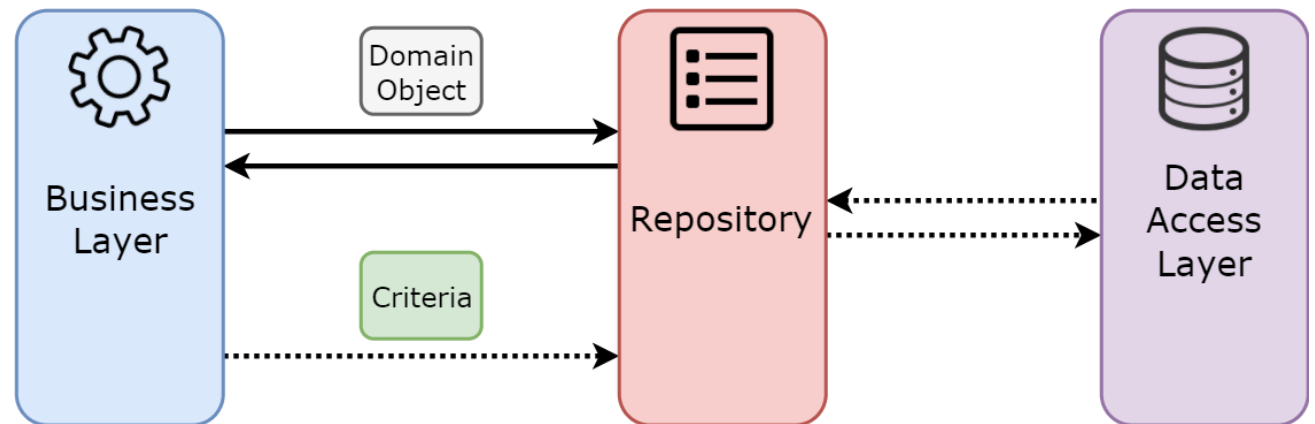
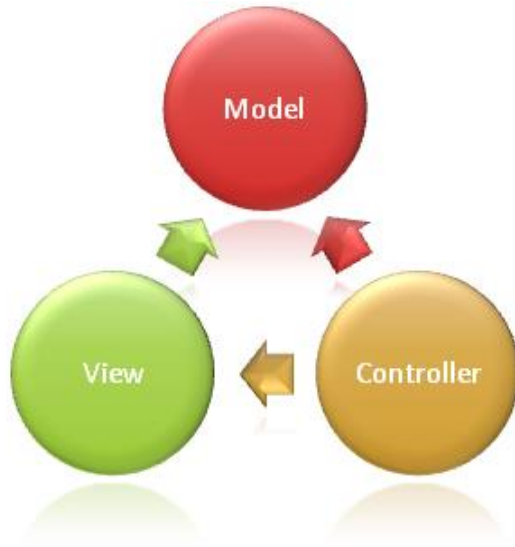
SOLID Design Principles



TDS: Competencias (Teoría y Prácticas)

■ Introducción a la noción de arquitectura software

- Conocer patrón arquitectural “Layers” para organizar una aplicación en **varias capas**
- Conocer el concepto de Arquitectura Limpia
- Aplicación de la **arquitectura MVC** de tres capas: *Presentación*, *Dominio* o *Lógica de Negocio* y *Almacenamiento*. Uso de un *controlador* que separa la presentación de las clases del negocio. Uso del patrón Repositorio para acceso a bases de datos.



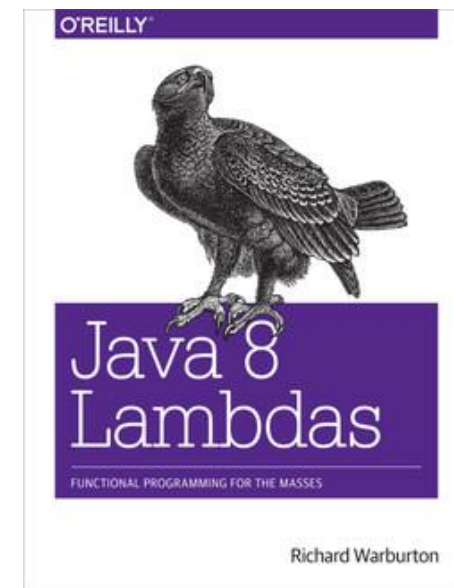
TDS: Competencias (Teoría)

■ Conceptos de programación OO

- Expresiones lambda
- Manejo de Streams
- Introspección en Java
- Clases Internas

■ Patrones de diseño

- Los patrones de diseño como forma de reutilizar la experiencia.
- Patrones de creación, estructurales y de comportamiento.
- Adquirir destreza en la identificación de los patrones aplicables a un determinado problema.
- Aplicar patrones en el diseño e implementación de una aplicación.



Design Patterns

Elements of Reusable
Object-Oriented Software

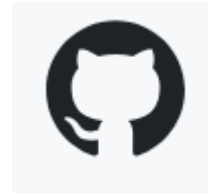
Erich Gamma
Richard Helm
Ralph Johnson
John Vlissides



Foreword by Grady Booch

TDS: Competencias (Prácticas)

- **Uso de un depurador de código y sistema de log**
 - Depuración sobre eclipse
- **Herramientas para la gestión del desarrollo de software**
 - Comprender la utilidad de los **sistemas de control de versiones** (CVS)
 - Utilizar **GIT** en un proyecto de desarrollo de software en equipo (GitHub)



GitHub

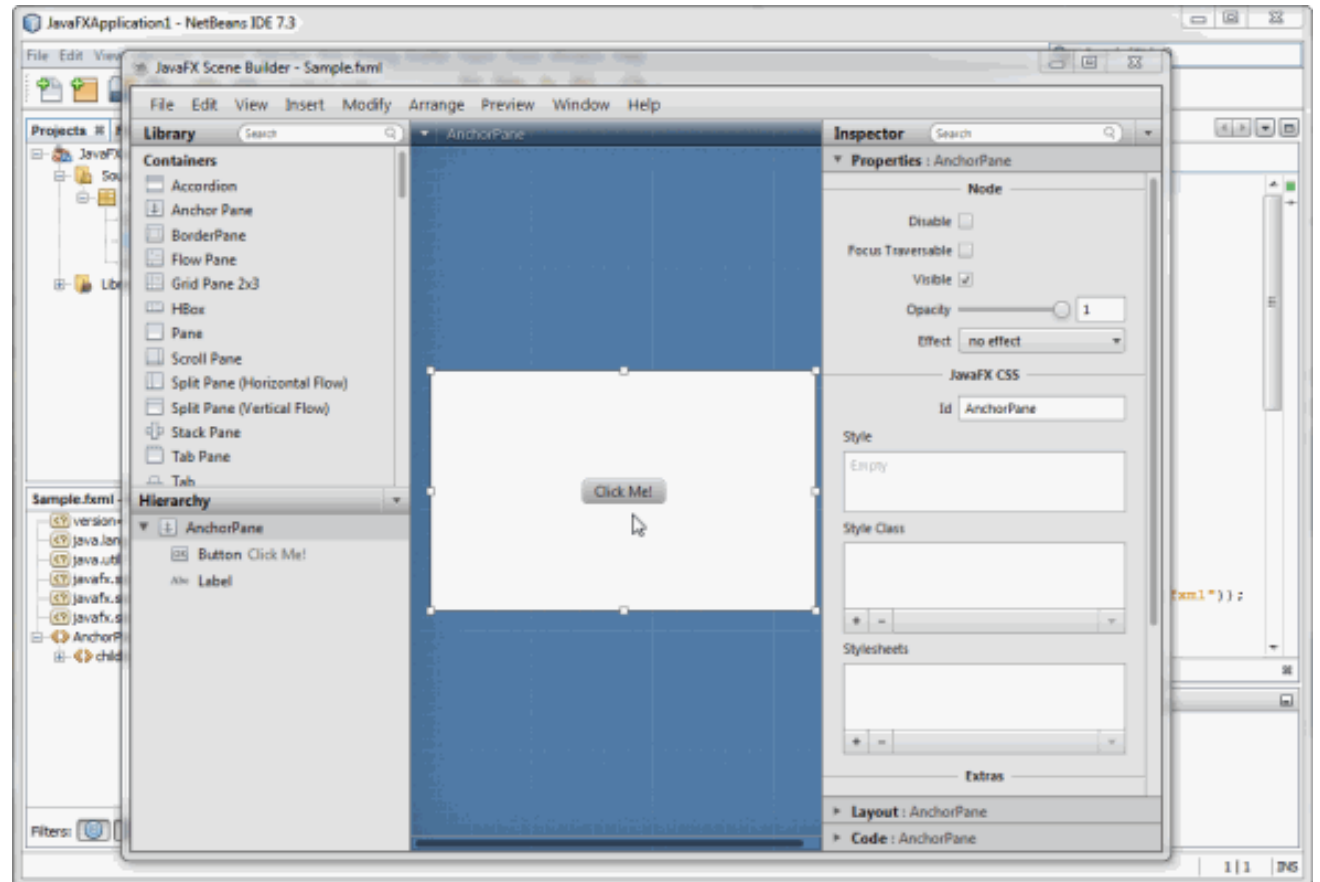
- Comprender la utilidad de las **herramientas de automatización de la construcción** de software (build tools).
- Utilizar **Maven** como build tool.

maven

TDS: Competencias (Prácticas)

- **Desarrollo de Interfaces Gráficas de Usuario (GUI)**

- Tecnología GUI de escritorio para Java: **JavaFX**
- Uso de *herramienta* de construcción de GUIs (**Scene Builder**)



Clases de Teoría (26 horas)

Teoría

- Tema 1. Introducción al Desarrollo de Software (2 horas)
- Tema 2. Principios básicos de construcción de software OO (3 horas)
- Tema 3. Conceptos en programación OO (4 horas)
- Tema 4. Patrones de diseño (8 horas)

Ejercicios y Talleres

- 1. Ejercicios Patrones GRASP (2 hora)
- 2. Ejercicios Programación Funcional en Java 8 (2 horas)
- 3. Ejercicios Patrones de Diseño (4 horas)

Clases de Prácticas (12 sesiones)

Prácticas

Práctica 1. Git (1 sesión)

Práctica 2. Maven (1 sesión)

Práctica 3. Depuración y Logging(1 sesión)

Práctica 4. Arquitectura de la aplicación (1/2 sesión)

Práctica 4-7. Desarrollo de interfaces de usuario con JavaFX (3 + 1/2 sesiones)

Práctica 8. Persistencia de datos (1 sesión)

Práctica 9-10. Implementación de patrones de diseño(2 sesiones)

Talleres

1. Taller 1: Requisitos (1 sesión)

2. Taller 2: Modelado con UML (1 sesión)

Planificación temporal aproximada

Semana	Teoría	Laboratorio
1	PRESENTACIÓN + T1 (INTRODUCCIÓN)	
2	T1 + T2 (GRASP, PRINCIPIOS)	Git
3	T2	Maven
4	EJERCICIOS GRASP	Taller Requisitos
5	T3 (Clases Anidadas, Streams y Expresiones Lambda, Introspección)	Taller Modelado
6	T3	Depuración + Loogin
7	EJERCICIOS STREAMS	MVC + GUI: JavaFX
8	T4 (PATRONES DISEÑO - CREACIÓN)	GUI: JavaFX
9	T4 (PATRONES DISEÑO - ESTRUCTURALES)	GUI: JavaFX
10	T4 (PATRONES DISEÑO - ESTRUCTURALES) + T4 (PATRONES DISEÑO - COMPORTAMIENTO)	GUI: JavaFX
11	T4 (PATRONES DISEÑO - COMPORTAMIENTO)	Persistencia
12	EJERCICIOS PATRONES DE DISEÑO	Patrones de Diseño
13	EJERCICIOS PATRONES DE DISEÑO	Patrones de Diseño

Evaluación

Teoría

- Examen **(14 de diciembre, mañana)** con cuestiones teórico-prácticas del mismo tipo que las propuestas en boletines de ejercicios vistos en clase.

Prácticas

- **Proyecto de programación**, cuya especificación se entregará en las primeras semanas del curso.
 - Los alumnos formarán **grupos de tres**.
 - **IMPORTANTE**: Deben pertenecer al **mismo grupo de teoría**.
 - **Entrega final del proyecto (7 de enero)**
- **Examen de programación** en los laboratorios, donde se requerirá al alumno poner en práctica las enseñanzas de la asignatura.

Evaluación

- Para superar la asignatura los alumnos deberán **haber aprobado tanto la teoría como las prácticas (nota ≥ 5 en cada parte)**.
- En la parte de teoría el instrumento de evaluación será un examen escrito
- En la parte de prácticas, los instrumentos de evaluación serán dos:
 - Un proyecto implementado en grupo y entregable que supone el 20% de la nota de prácticas (el profesorado puede o no solicitar una entrevista de defensa)
 - Un examen práctico en ordenador que supone el 80% de la nota de prácticas
 - El proyecto no es de entrega obligatoria, y en caso de no entregar el alumno perderá 2 de los 10 puntos de la nota de prácticas
 - El examen práctico en ordenador si es de obligatoria presentación. Su calificación (hasta 8 en la nota de prácticas) se sumará a la nota del proyecto, siendo necesario obtener un 5 para superar la parte práctica.
- En la nota final, la parte de teoría y del proyecto **tendrán ambos un peso del 50%**.
- Una vez aprobada una parte se mantiene la nota en las siguientes convocatorias del curso actual, **pero no para el próximo curso**.

Carga de trabajo

- Clases de teoría: 26 horas
- Clases prácticas: 20 horas
- Estudio asignatura : 50 horas
- Dedicación Proyecto (preparación examen práctico): 54 horas

Total: 150 horas (6 ECTS x 25 horas)

Bibliografía básica

■ Temas 1 y 2

- **Craig Larman**, *UML y Patrones*, 2ª edición, Prentice-Hall, 2002

- Tema 3

- Richard Warburton, *Java 8 Lambdas: Functional Programming For The Masses*, O'Reilly, 2014
- The Java Tutorial: Lambda Expressions
<https://docs.oracle.com/javase/tutorial/java/javaOO/lambdaexpressions.html>

■ Tema 4

- **Erich Gamma et al.**, *Patrones de Diseño*, Addison-Wesley, 2002.
- Mark Grand, *Patterns in Java*, vol. 1, John Wiley, Segunda Edición, 2002.
- Eric T. Freeman et al. “Head First Design Patterns”, O'Reilly, 2004.



ChatGPT



Bibliografía básica

■ Git

- *Pro Git*, Scott Chacon y Ben Straub, Apress, <https://git-scm.com/book/en/v2>
- *Eclipse Git Tutorial*, Lars Vogel, 2021, <https://www.vogella.com/tutorials/EclipseGit/article.html>

- Maven

- *Tutorial Maven*, Jenkov.com, 2021 <https://jenkov.com/tutorials/maven/maven-tutorial.html>.
- *Using Maven within the Eclipse IDE – Tutorial*, Lars Vogel, 2021, <https://www.vogella.com/tutorials/EclipseMaven/article.html>

■ JavaFX: Interfaces de Usuario

- OpenJFX: <https://openjfx.io/>